```
# 2
# import numpy as np
# from scipy.linalg import lu

# A = np.array([[1, 1, 1],
#               [4, 3, -1],
#               [3, 5, 3]], dtype=float)

# B = np.array([1, 6, 4], dtype=float)

# P, L, U = lu(A)

# Y = np.linalg.solve(L, np.dot(P, B))
# X = np.linalg.solve(U, Y)

# print("Solution (x, y, z):")
# print(X)

                        # 1
# import numpy as np


# A = np.array([[2, 1, 3],
#               [1, 2, 1],
#               [3, 1, 2]], dtype=float)

# A_inv = np.linalg.inv(A)

# print("Matrix A:")
# print(A)

# print("\nInverse of A:")
# print(A_inv)
#3
import numpy as np

# A = np.array([[4, -1, 1],
#               [2, 5, -1],
#               [-1, 1, 4]], dtype=float)

# B = np.array([3, 1, 2], dtype=float)

# # x = np.zeros(3)
# n = 10

# print("Jacobi Method:")
# for k in range(n):
#     x_new = np.zeros(3)
#     for i in range(3):
#         s = sum(A[i][j] * x[j] for j in range(3) if j != i)
```

```
#         x_new[i] = (B[i] - s) / A[i][i]
#     x = x_new
#     print(f"Iteration {k+1}: {x}")

# 3b
# x = np.zeros(3)

# print("\nGauss-Seidel Method:")
# for k in range(n):
#     for i in range(3):
#         s = sum(A[i][j] * x[j] for j in range(3) if j != i)
#         x[i] = (B[i] - s) / A[i][i]
#     print(f"Iteration {k+1}: {x}")
# 4
# import numpy as np

# A = np.array([[4, -2, 1],
#               [1,  1, -1],
#               [3, -2, 2]], dtype=float)

# # initial guess
# x = np.array([1, 1, 1], dtype=float)

# n = 10  # number of iterations

# for i in range(n):
#     x_new = np.dot(A, x)

#     # dominant eigenvalue approximation
#     eigenvalue = max(abs(x_new))

#     # normalize eigenvector
#     x = x_new / eigenvalue

#     print(f"Iteration {i+1}")
#     print("Eigenvalue ≈", eigenvalue)
#     print("Eigenvector ≈", x)
#     print()
# import numpy as np

A = np.array([[2, 1, 3],
              [1, 2, 1],
              [3, 1, 2]], dtype=float)

n = len(A)

# Create augmented matrix [A | I]
I = np.identity(n)
aug = np.hstack((A, I))

```

```python
# Gauss-Jordan elimination
for i in range(n):
    # Make diagonal element 1
    diag = aug[i][i]
    for j in range(2*n):
        aug[i][j] /= diag

    # Make other elements in column zero
    for k in range(n):
        if k != i:
            factor = aug[k][i]
            for j in range(2*n):
                aug[k][j] -= factor * aug[i][j]

# Extract inverse matrix
A_inv = aug[:, n:]

print("Matrix A:")
print(A)

print("\nInverse of A (without inbuilt function):")
print(A_inv)
```